

Outline

Command at the Speed of Thought: natural-language-commanded strategy games under real-time compute budgets.

1. **Vision:** play a strategy game the way a commander works, by speaking.
2. **The research question:** what does language command cost at game speed?
3. **Prior work and the open gap.**
4. **The program:** three layers of research, four rungs of environments.
5. **Phase 1:** the first benchmark paper.
6. **Architecture, reinforcement learning, papers, collaboration, risks.**

Vision: the Commander Interface

In a real-time strategy (RTS) game, most player effort is micromanagement: drag-selecting units, hunting icons, sustaining hundreds of actions per minute. The strategic thinking that makes the genre rewarding gets a fraction of the player's attention. The rest is manual labour forced by the interface.

The alternative division of work already exists in coding agents: the person speaks intent, agents execute, the person reviews and redirects. Applied to strategy games, the player becomes a commander: **watch, listen, think, speak; subordinates execute.**

- The interface concept shipped in 2008: EndWar, a fully voice-commanded RTS, on a rigid 70-word grammar.
- Language models removed the grammar limit: free-form commands are now understood.
- What is still missing is **speed**: language models cannot yet act at game pace.



Figure 1: what the player actually does. StarCraft II through DeepMind's PySC2 interface: economy, production, and unit control all run in parallel against the clock.

Still: PySC2 mini-games video (DeepMind)

The Research Question

How much does natural-language command cost, in latency and memory, at game speed, and how can that cost be driven down?

The evaluation philosophy, shared by every part of the program:

- Score systems by **performance as a function of latency budget and VRAM budget**: an efficiency frontier, not a single number.
- The clock never pauses. A decision that arrives late is a **no-op** ("no operation": it simply has no effect, exactly as a move you thought of too late has no effect in a real match).

This is the question that decides whether the commander interface is buildable at all.

Prior Work, and the Gap

LLMs already play RTS through text. TextStarCraft II (NeurIPS 2024): off-the-shelf models beat the level-5 built-in AI with chain-of-summarization prompting. Adaptive Command (arXiv 2025), adds a speech-interfaced strategic advisor for human players. Both sidestep real time by slowing or pausing the clock.

Efficiency is the named open problem. The survey of LLM game agents (ACM Computing Surveys 2026) devotes §8.1, "Action Games: Low-Latency Response," to it: "the core challenge is low-latency response, which shapes agent design by requiring fast action and hybrid architectures that reconcile LLM reasoning with frame-level responsiveness." A 1.3M-parameter specialist (arXiv 2026) beats LLMs up to 92,000 times larger at real-time DOOM (the classic 1993 first-person shooter, a standard real-time benchmark), deciding in 31 ms.

Efficiency methods are never evaluated where stakes are real. KV-cache eviction (dropping old context to fit memory) is scored on perplexity and retrieval suites. No existing work evaluates eviction closed-loop inside a live game, where evicting the wrong memory loses the match minutes later.

The gap, in one line: a streaming game is the natural stress test for efficient-inference methods, and nobody has run it. The gap is named in a public survey; speed decides who fills it.

The Program: Three Layers

Layer	Function	Status
Strategy	An LLM commander reads the game-state stream and issues macro decisions under hard latency and KV-cache budgets	Phase 1
Foresight	A compact entity-level world model answers the commander's counterfactual queries: "if the army pushes now, does the fight win?"	Phase 2 to 3
Embodiment	Units execute commands with model-generated motion at crowd scale on a consumer GPU (generated: synthesized for the command by a motion model, not replayed animation clips)	Phase 2 to 3

Each layer is a publishable line of work; all share the budget-frontier evaluation.

The Roadmap: Four Rungs, One Harness

Harness: the shared scaffolding every rung runs inside: command protocol, unpausable clock, deadlines, metric logging.

Rung	Environment	What it isolates
0	Command arena: colour-tagged agents in a room move in discrete directions under streamed commands (Figure 2)	Per-entity command grounding against the clock (grounding: mapping words to the right agent and action)
1	StarCraft II, clock unpaused	Strategic judgment under latency and KV-cache budgets: the Phase 1 paper
2	Multi-user arena: several players command their own units in one shared real-time world	Efficiency as competitiveness: does a fast small commander beat a slow large one?
3	The commander game	Product north star, beyond the paper horizon

The end-state game is a vision, not a deliverable: it fixes the two constraints every paper inherits: unpausable clock, hard compute budget. The harness, command protocol, and metrics carry up the ladder.

Rung 0 Up Close: the Command Arena

Streamed commands

t = 0.0 s **"Gray, walk south."**

t = 1.2 s **"Yellow, sit down."**

t = 2.1 s **"Blue, run north."**

Difficulty rises along three axes:

command rate, compositional addressing

("everyone except the yellow one..."),

and commands about remembered state

("the one who was sitting earlier...").

*The clock never pauses: a late action
is a no-op.*

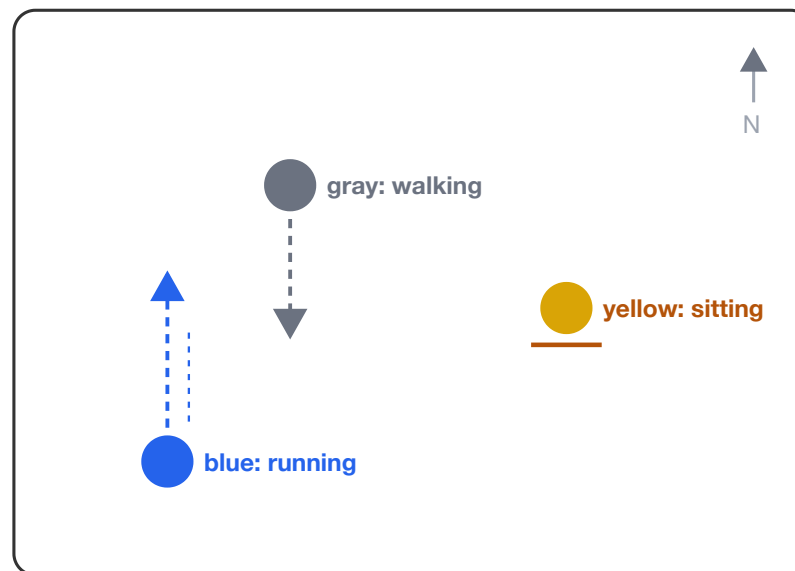


Figure 2: the command arena. Colour-tagged agents under a streamed command clock; each command is colour-keyed to its agent. Single commands are trivially easy for any modern model, by design: the test is the stream. Measured: grounding accuracy, utterance-to-action latency, and deadline misses, as command rate rises.

Phase 1: the Wedge Benchmark

The wedge: a deliberately narrow, fast first paper that opens the agenda behind it. Here, a benchmark and empirical study: **can an LLM command at game speed, and which efficiency techniques preserve its judgment?** Built on the TextStarCraft II stack, with the clock unpaused, wall-clock decision deadlines, and a VRAM ceiling.

Experimental variables

- KV-cache policy: full cache, StreamingLLM sinks, H2O, SnapKV, OBCache (ICML 2026)
- Model scale and sparsity: 1B to 70B open-weight models, structurally pruned and quantized variants
- Architecture: monolithic commander vs a commander/executor split
- State encoding: plain text vs a learned tokenizer over structured game state

Metrics: win rate against fixed built-in AI levels as a function of (latency budget, VRAM budget); actions per minute, deadline misses, tokens per decision.

Strategic-memory probes: scripted scenarios where winning requires recalling information observed minutes earlier. They convert cache-eviction failures from abstract retrieval misses into observable lost games.

Architecture: Slow Thinker, Fast Actors

The variable most likely to matter: **monolithic commander vs a commander/executor split**, a slow strategic model directing fast small executors.

Two independent lines of support:

- The DOOM result: a 1.3M-parameter specialist decides in 31 ms while far larger models miss the deadline. Small executors can hold the clock.
- Robotics arrived at the same shape: $\pi 0$ (arXiv 2024) pairs a slow vision-language backbone with a fast action expert controlling at up to 50 Hz. A game is the cheap, safe place to iterate on that split.

State tokenization: game state is a structured, non-text modality (entities, positions, events; Figure 3). Following Graph Tokenization (ICLR 2026), train a byte-pair tokenizer on replay corpora; measure tokens per decision and latency at equal win rate. A compact state code shrinks every downstream cost.

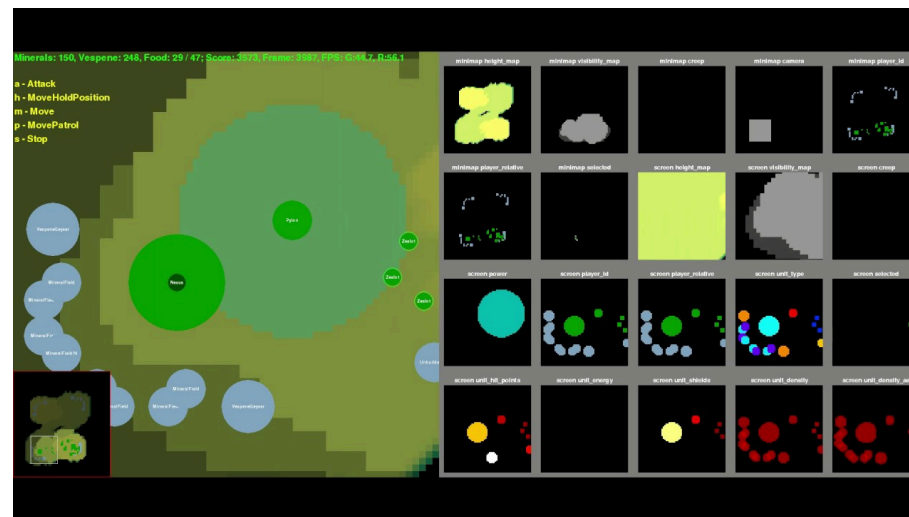


Figure 3: game state is not text. PySC2 exposes StarCraft II as entity and terrain feature layers (right grid); serializing this to prose is the naive baseline the learned tokenizer competes against.

Still: PySC2 video (DeepMind)

Where This Sits Relative to Reinforcement Learning

Dr. Diao's framing: "essentially RL in a virtual world." Precisely scoped:

- **The substrate is RL.** PySC2 is a reinforcement-learning environment; win rate is a reward signal; the strategic-memory probes are a credit-assignment test (which earlier decision caused the eventual outcome?).
- **Phase 1 trains nothing.** Off-the-shelf and pruned models, prompted. The novelty is the efficiency frontier, which no RL benchmark measures.
- **The long arc is RL.** Learned executors, finetuned commanders, and the foresight world model arrive in Phase 2 to 3.
- **The reconciling frame is real-time RL:** the environment does not pause and thinking time has a cost. This benchmark puts latency inside the decision problem.
- A nuance from robotics: π_0 , the suggested architecture analogy, is itself imitation-trained (it learns from demonstrations, not from rewards). The near-term path is supervised; RL depth arrives with the later phases.

The Papers This Produces

Paper	The question it answers	Who cares, independent of the game
Real-time commander benchmark (Phase 1)	Which efficiency methods survive a closed-loop game clock?	KV-cache and pruning researchers: their methods are never scored by win rate
Game-state tokenizer	Does byte-pair tokenization extend to entity and event streams?	The tokenization-beyond-text programme
Command-arena benchmark (rung 0)	How does per-entity grounding degrade as command rate rises?	Real-time agent and interactive-systems researchers
Competitive-efficiency study (rung 2)	Does a fast small commander beat a slow large one? Rating vs compute budget	Inference-efficiency and agents communities
Crowd motion under budget (embodiment)	Can language-commanded full-body crowds run in real time on one GPU?	Motion-generation and graphics community

The test each paper must pass: it answers a question its community already cares about, while caring nothing about the game.

Collaboration, Risks, Open Questions

Why this collaboration. The wedge needs two things at once: the efficiency-methods stack (cache eviction, structured pruning, tokenization beyond text), where Dr. Diao's recent work supplies methods and baselines, and benchmark engineering plus genuine RTS fluency. The combination is rare: agent-interface groups lack the efficiency depth; efficiency groups evaluate on static corpora.

Risks

- Environment engineering is real work: mitigated by inheriting the TextStarCraft II stack.
- The gap is named in a public survey and could be filled by others: the first paper rewards being first more than being perfect.
- Latency results age as inference gets cheaper: the framing, performance vs budget, remains meaningful regardless of which model currently wins.

Open questions: venue and timing (ICLR vs NeurIPS 2027); whose GPUs run the frontier sweep; tokenizer inside the first paper or standalone; DreamSoul product interest beyond the papers.

Full proposal, decisions log, and this deck: `DreamSoul-AI/game-commander`.